

Analysis and Synthesis of Algorithms

Design of Algorithms

Backtracking and Bounding

Copyright 2026, Pedro C. Diniz, all rights reserved.

Students enrolled in the Design of Algorithms class at Faculdade de Engenharia da Universidade do Porto (FEUP) have explicit permission to make copies of these materials for their personal use.

Overview

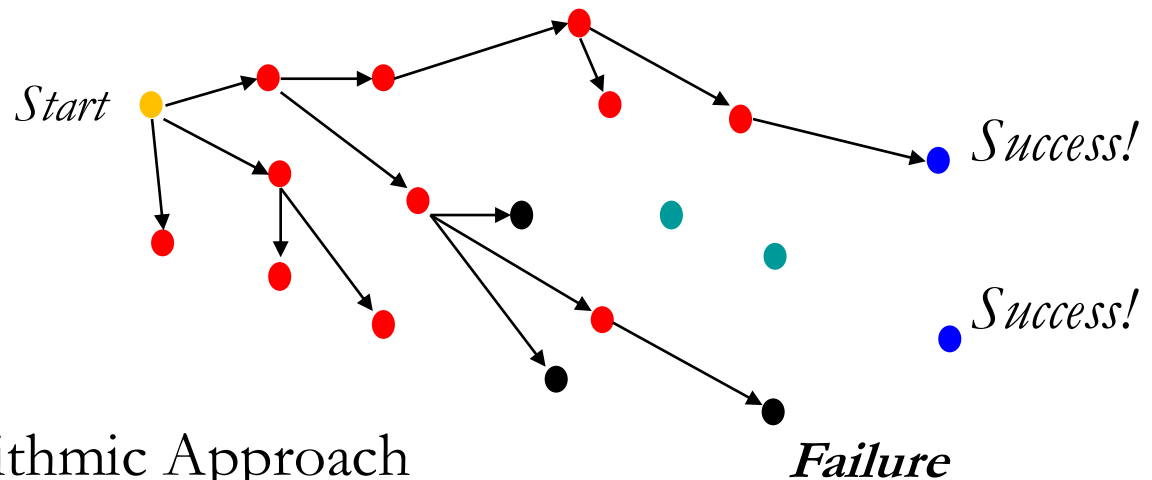
- Why Backtracking?
- Motivation: Sum of SubSets
- Backtracking Algorithm Formulation
- Bounding & Pruning for Effectiveness
- Other Example Problems

Why Backtracking?

- Suppose you have to make a series of *Decisions*, among various *Choices*, where:
 - You don't have enough information to know what to choose;
 - Each decision leads to a new set of choices;
 - Some sequence of choices (possibly more than one) may be a solution to your problem;
- Backtracking is a methodical way of trying out various sequences of decisions, until you find the correct one that “works”

Why Backtracking?

- Targets: Class of Search Problems
 - Large Space of Configuration or States
 - Defined as a Sequence of Choices
 - Search for Optimal Solution(s)
 - Satisfying some Constraints



- Basic Idea:
 - Simple Algorithmic Approach
 - Systematically, Exhaustively Search the “Solution” Space
 - Collect Best Solution

Motivation: Sum of Subsets

- Problem:
 - Given $n + 1$ positive numbers: w_i , $1 \leq i \leq n$ and M ,
Finding all subsets of the W ; whose sum is ***exactly*** M .
- Example:
 - $n = 4$, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$ and $M = 31$
 - Solutions:
 - $(w_1, w_2, w_4) = (11, 13, 7)$ and $(w_3, w_4) = (24, 7)$ or
 - Tuple of fixed size of element indices: $(1, 1, 0, 1)$ and $(0, 0, 1, 1)$

Motivation: Sum of Subsets

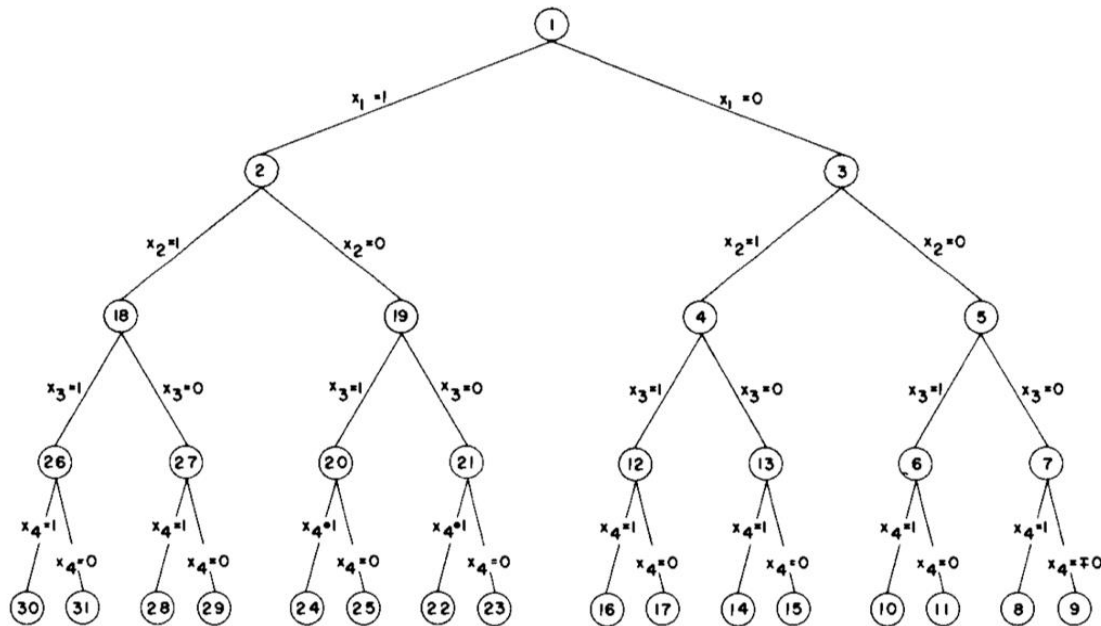
- Easy Algorithm:
 - Enumerate all Subsets
 - Compute the Sum of each
 - Select the ones that add up to M
- Number of Subsets of a Set with Cardinality N?
 - 2^N , which can be large for large N
- Better Approach:
 - Explore Sequence of Choices
 - Exhaustively, using Recursion or Iteration
 - Avoid Exponential Exploration by Skipping unpromising nodes

Motivation: Sum of Subsets

- Sum of Subsets:
 - Sequentially, make the choice of including/excluding each item in the addition.
 - Explore all choices and backtrack if each specific choice was bad.
 - Implicitly defines a Tree of Choices or States:

Motivation: Sum of Subsets

- Sum of Subsets:
 - Sequentially, make the choice of including/excluding each item in the addition.
 - Explore all choices and backtrack if each specific choice was bad.
 - Implicitly defines a Tree of Choices or States:



Sum of Subsets: Recursive Formulation

- Recursive Search Formulation:

```

procedure RecursiveBackTracking(real X[1:n], integer k)
  global integer n, real X[1:n];
  for each X[k] possible choice in Choices(X[1:k-1]) do
    if Solution(X[1:k]) = true then
      return X[1:k];
    else
      call RecursiveBackTracking(X[1:n], k+1)
    endif
  end for
end
  
```

Sum of Subsets: Recursive Formulation

- Recursive Search Formulation:

```

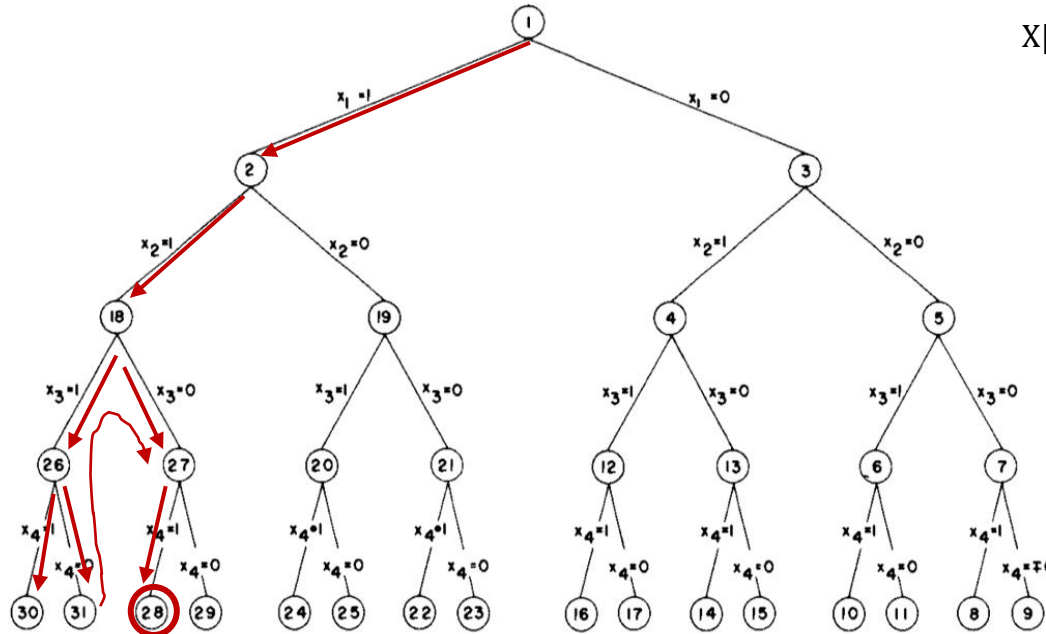
procedure RecursiveBackTracking(real X[1:n], integer k)
  global integer n, real X[1:n];
  for each X[k] possible choice in Choices(X[1:k-1]) do
    if Solution(X[1:k]) = true then
      return X[1:k];
    else
      call RecursiveBackTracking(X[1:n], k+1)
    endif
  end for
end
  
```

Explores all Choices at this Level given Previous choices

Check if the Sequence of Choices so far is a Solution

Explores next Level Down

Sum of Subsets: Tracing Recursive Calls



RecursiveBackTracking(X, 1)

$X[1] = 1$, $\text{Cost}(1,0,0,0) = 11$

RecursiveBackTracking(X, 2)

$X[2] = 1$, $\text{Cost}(1,1,0,0) = 24$

RecursiveBackTracking(X, 3)

$X[3] = 1$, $\text{Cost}(1,1,1,0) = 48$

RecursiveBackTracking(X, 4)

$X[4] = 1$, $\text{Cost}(1,1,1,1) = 55$

$X[4] = 0$, $\text{Cost}(1,1,1,0) = 48$

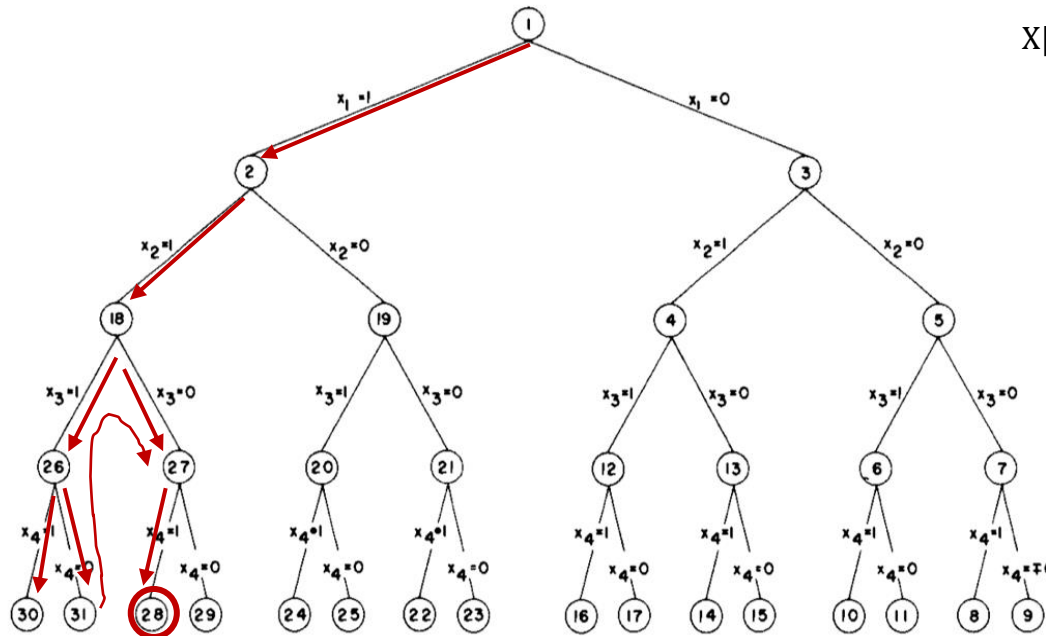
$X[3] = 0$, $\text{Cost}(1,1,0,0) = 24$

RecursiveBackTracking(X, 4)

$X[4] = 1$, $\text{Cost}(1,1,0,1) = 31$

Optimal Solution

Sum of Subsets: Tracing Recursive Calls



RecursiveBackTracking(X, 1)

$X[1] = 1$, $\text{Cost}(1,0,0,0) = 11$

RecursiveBackTracking(X, 2)

$X[2] = 1$, $\text{Cost}(1,1,0,0) = 24$

RecursiveBackTracking(X, 3)

$X[3] = 1$, $\text{Cost}(1,1,1,0) = 48$

RecursiveBackTracking(X, 4)

$X[4] = 1$, $\text{Cost}(1,1,1,1) = 55$

$X[4] = 0$, $\text{Cost}(1,1,1,0) = 48$

$X[3] = 0$, $\text{Cost}(1,1,0,0) = 24$

RecursiveBackTracking(X, 4)

$X[4] = 1$, $\text{Cost}(1,1,0,1) = 31$

Optimal Solution

Maybe we wanted to collect all optimal solution, so we need to proceed...

Sum of Subsets: Bounding

- **Observations:**

- If at a given state, adding all remaining elements will not get to M , then the choice taken so far are infeasible.

$$\sum_{i=1}^k W(i)X(i) + \sum_{i=k+1}^n W(i) < M$$

- If values are sorted, and if at a given state, if adding the next element will lead to a value greater than M .

$$\sum_{i=1}^k W(i)X(i) + W(k+1) > M$$

- Bounding Function: If **false**, skip exploring current tree node, if **true** continue exploring nodes

$$B_k(X(1), \dots, X(k)) = \text{true} \text{ iff } \left(\sum_{i=1}^k W(i)X(i) + \sum_{i=k+1}^n W(i) \geq M \right.$$

$$\left. \text{and } \sum_{i=1}^k W(i)X(i) + W(k+1) \leq M \right)$$

Sum of Subsets: Recursive w/ Bounding

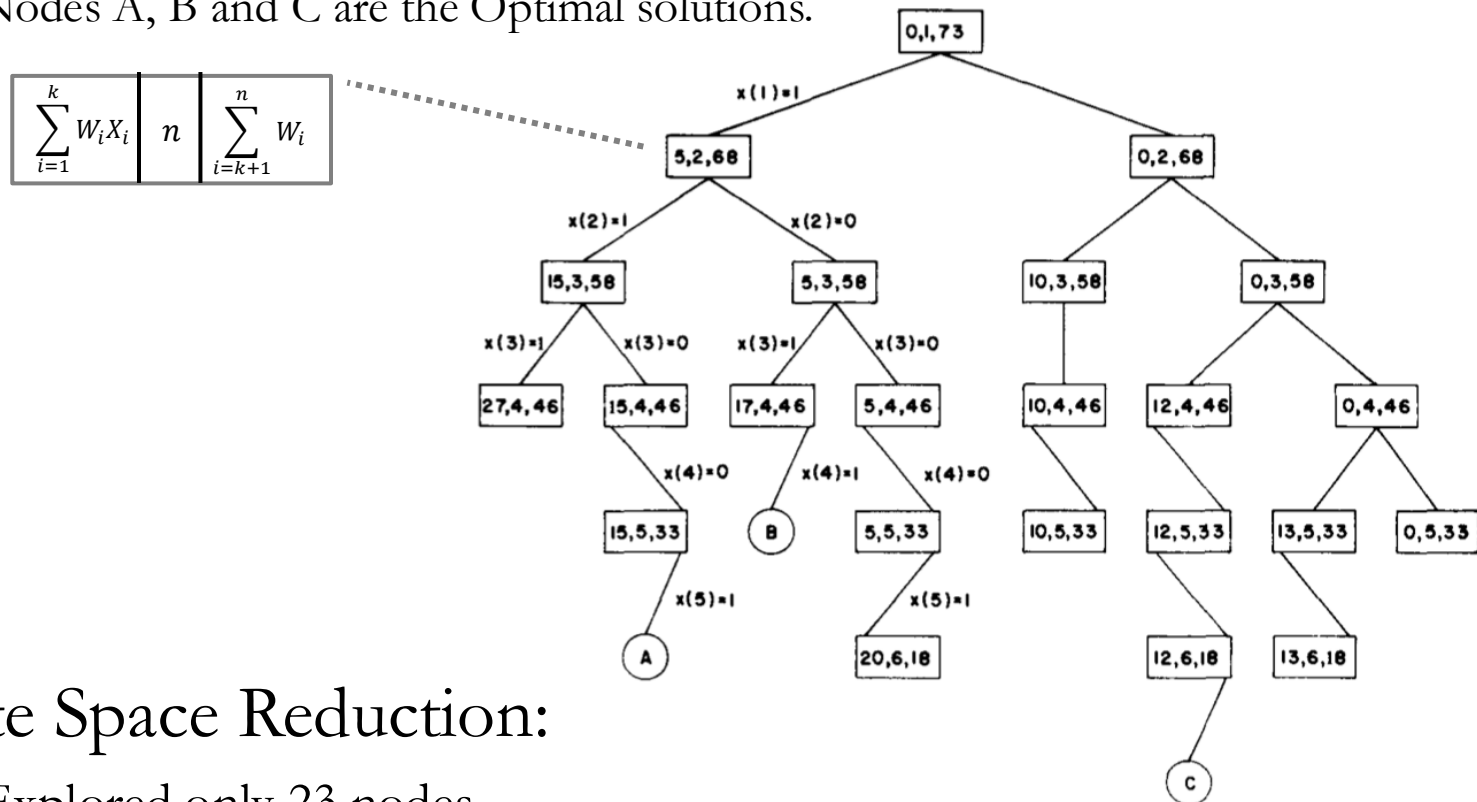
- Recursive Search Formulation:

```

procedure RecursiveBackTracking(real X[1:n], integer k)
  global integer n, real X[1:n];
  for each X[k] possible choice in Choices(X[1:k-1])
    and Bound(X[1:k]) = true do
      if Solution(X[1:k]) = true then
        return X[1:k];
      else
        call RecursiveBackTracking(X[1:n], k+1)
      endif
    end for
  end
  
```

How Effective is Bounding?

- Example: $n = 6$, $M = 30$ and $W[1:6] = \{5, 10, 12, 13, 15, 18\}$.
 - Circular nodes represent points at which a subset with sum M is printed out.
 - Nodes A, B and C are the Optimal solutions.



- State Space Reduction:
 - Explored only 23 nodes.
 - Full Space contains $2^6 - 1 = 63$ (this count excludes 64 leaf nodes).

Sum of Subsets: Iterative Formulation

- Iterative Search Formulation with Bounding:

```

procedure IterativeBackTracking(real X[1:n], integer n)
  integer k, n, local real X[1:n];
  k = 1;
  while (k > 0) do
    if there is an untried X[k] in Choices(X[1:k-1])
      and Bound(X[1:k]) = true then
        if Solution(X[1:k]) = true then
          save solution X[1:k];
        endif
        k = k + 1;
      else
        k = k-1;
      endif
    end for
  end

```

Backtracking Formulation

- Sequence of Decisions forms a Tree of Decisions or States resulting from each Decision.
- States must be expressible as an n-tuple $(x_1, \dots, x_n)$ where the x_i are chosen from some Finite set S
- All paths from the Root to other nodes define the State Space of the Problem.
- Often the Problem to be Solved calls for finding one vector which Maximizes (or Minimizes / Satisfies) a Criterion Function $P(x_1, \dots, x_n)$
- Constraints determines which States are Feasible

$$\begin{array}{ll}
 x_i \geq 0 & \text{or } S_i = \{\text{all nonnegative real numbers}\} \\
 x_i = 0 \text{ or } 1 & \text{or } S_i = \{0, 1\} \\
 l_i \leq x_i \leq u_i & \text{or } S_i = \{a: l_i \leq a \leq u_i\}
 \end{array}$$

Bounding & Prunning for Effectiveness

- Suppose m_i is the Size of Set S_i .
 - Then there are $m = m_1 \times m_2 \times \dots \times m_n$ n-tuples which are possible Feasible and Optimal nodes.
- Brute Force Approach
 - Form all of these m n-tuples and evaluate each one with P , saving those which yield the Optimum Value.
- Implementation: Pruning using Bounding Function
 - Recursive or iteratively, implicitly create a Tree of States
 - Skip exploration of tree nodes (partially filled vectors) if they are not promising or lead to unpromising solutions.
- In Practice,
 - Very efficient with the appropriate Bounding Functions.

Other Example Problems

- Graph Coloring:
 - Let G be a graph and m be a given positive integer. We want to discover if the nodes of G can be colored in such a way that no two adjacent nodes have the same color yet only m colors are used.
- Hamiltonian Cycles:
 - Let $G = (V, E)$ be a connected graph with n vertices. A Hamiltonian cycle is a round trip path along n edges of G which visits every vertex only once and returns to its starting position.
- 0/1 Knapsack:
 - Given N positive weights w_i , N positive profits p_i , and a positive number M which is the Knapsack capacity, this problem calls for choosing a subset of the weights such that

$$\sum_{1 \leq i \leq n} w_i x_i \leq M \quad \text{and} \quad \sum_{1 \leq i \leq n} p_i x_i \text{ is maximized}$$

The x 's constitute a zero-one valued vector.

Summary

- Why Backtracking?
- Motivation: Sum of SubSets
- Backtracking Algorithm Formulation
- Bounding & Pruning for Effectiveness
- Other Example Problems